Data Structures

Data structures

Joseph Mhango

2024-09-13

Objectives

By the end of this section, you should be able to:

- Identify and describe basic data types in R.
- Understand how factors represent categorical data.
- Use `class()` and conversion functions to change variable types.
- Work with vectors, matrices, and lists.
- Apply best practices in naming and managing R objects.

Basic data types in R

R works with different kinds of **data objects**, and each object has an associated **type**. These types determine what operations are possible and how the data is treated during analysis. All the data you create lives in the **Global Environment**, and it's important to know what kind of data you're working with.

R is pretty good at guessing what type your data should be — but this can sometimes backfire. Think of R as a *passive-aggressive butler*: it tries to help, but doesn't always tell you when it's done something "helpful." So, you should learn to check and control data types explicitly.

Basic primitive types in R

Here are the most common **primitive data types**:

```
a <- 1    # numeric
mode(a)
```

```
[1] "numeric"
```

```
a <- "1"  # character
mode(a)
```

```
[1] "character"
```

```
a <- '1'  # character
mode(a)
```

```
[1] "character"
```

```
a <- TRUE # logical
mode(a)
```

```
[1] "logical"
```

```
a <-  1i  # complex
mode(a)
```

```
[1] "complex"
```

These types include: - **numeric**: Numbers like 1, 3.14 - **character**: Strings like “apple” - **logical**: TRUE or FALSE - **complex**: Numbers with imaginary parts like 1i

---

Special values in R

There are a few special types of values that you'll come across often:

- `NULL` means an undefined or empty object (a missing vector). Use `is.null()` to test it.
- `NA` stands for *Not Available* and represents missing data. Check with `is.na()`.
- `Inf` and `-Inf` mean positive or negative infinity (e.g., from `1/0`). Use `is.infinite()`.
- `NaN` means *Not a Number*, e.g. `0/0`. Use `is.nan()`.

```
my_list <- list(a = 1, b = 2)
my_list <- my_list$c  # Element 'c' doesn't exist, Output: NULL

na_addition  <- 5 + NA
print(na_addition)  # Output: NA
```

```
[1] NA
```

Naming objects in R

There are some general rules and conventions for naming objects in R:

- Names must start with a letter.
- Names can include letters, numbers, underscores (_), and dots (.).
- Avoid special characters like `@`, `%`, `#`, etc.
- Avoid spaces — use underscores or camelCase instead.
- Use names that are short, descriptive, and consistent.

```
x1 <- 5

x2 <- "It was a dark and stormy night"

# harder to read and maintain:
my_variable_9283467 <- 1
```
